Lecture 7: TCP Congestion Control

Flow control versus congestion control

TCP limits how much unacknowledged data a sender may have in flight for two different reasons. **Flow control** protects the receiver: the receiver advertises a receive window (rwnd) with the amount of free buffer space it has, and the sender never exceeds it. **Congestion control** protects the network: the sender keeps its own congestion window (cwnd), an estimate of how much the path can carry without queues overflowing.

At any moment the sender may transmit up to the smaller of the two windows, so the effective window is min(cwnd, rwnd). A fast receiver on a slow, shared path is limited by cwnd; a slow receiver on a fast path is limited by rwnd.

Why congestion control exists

In October 1986 the early Internet suffered a series of congestion collapses: throughput between two sites at Berkeley fell from 32 kbit/s to 40 bit/s, because senders kept retransmitting into full queues. Van Jacobson's 1988 algorithms (slow start, congestion avoidance and fast retransmit) were the response, and every TCP implementation has used a descendant of them since.

Congestion signals

A TCP sender cannot see router queues directly. Classic TCP infers congestion from **packet loss**, detected either by a retransmission timeout or by duplicate acknowledgements. Newer algorithms also use **delay** (a growing round-trip time means queues are building) and explicit marks from routers (ECN).

Slow Start

A new connection does not know the capacity of the path, so it probes for it. During **slow start** the sender begins with a small initial window and increases cwnd by one maximum segment size (MSS) for every acknowledgement that arrives. Because a full window of segments is acknowledged each round-trip time, cwnd **doubles every RTT**: the growth is exponential, despite the name.

Initial window

Classic TCP started with an initial window of one segment. RFC 6928 (2013) raised the recommended initial window to ten segments, which lets short web transfers finish in fewer round trips. With an MSS of 1460 bytes, an initial window of ten segments is 14,600 bytes.

When slow start ends

Slow start continues until one of three things happens:

- cwnd reaches the **slow start threshold** (ssthresh), and the sender switches to congestion avoidance;
- a loss is detected, and the sender reacts as described in the section on loss recovery;
- cwnd reaches the receiver's window, so the receiver becomes the limit.

For a new connection ssthresh is set arbitrarily high, so the first slow start ends with the first loss. Afterwards ssthresh remembers half of the window at which loss occurred, and later slow starts stop there.

Worked example

Starting from ten segments and with no loss, cwnd is 20 segments after one RTT, 40 after two and 80 after three. On a path with a 100 ms RTT the sender reaches 80 segments (about 117 KB in flight) in 0.3 seconds.

Congestion Avoidance and AIMD

Once cwnd is at or above ssthresh the sender is near the capacity it last found, so it grows carefully. In **congestion avoidance** cwnd increases by MSS × MSS / cwnd for each acknowledgement, which adds roughly **one MSS per round-trip time**: linear rather than exponential growth.

Additive increase, multiplicative decrease

When loss signals congestion, the window is cut multiplicatively, classically halved. This combination is called **AIMD** (additive increase, multiplicative decrease). Plotted over time, cwnd forms the familiar TCP **sawtooth**: slow linear climbs ending in sharp drops.

AIMD is used because of its fairness property. Chiu and Jain showed in 1989 that when several flows share a bottleneck and all use AIMD, their windows converge towards equal shares of the capacity, whatever their starting points. Additive increase moves flows towards the efficiency line at the same rate, while multiplicative decrease shrinks large windows more than small ones.

AIMD in pseudo-code

```
on_ack():
    if cwnd < ssthresh:
        cwnd = cwnd + MSS              # slow start
    else:
        cwnd = cwnd + MSS * MSS / cwnd # congestion avoidance

on_triple_duplicate_ack():
    ssthresh = max(flight_size / 2, 2 * MSS)
    cwnd = ssthresh                    # multiplicative decrease
```

Loss Detection and Recovery

Retransmission timeout

The sender keeps a smoothed RTT estimate (SRTT) and its variation (RTTVAR). RFC 6298 sets the retransmission timeout to RTO = SRTT + max(G, 4 × RTTVAR), where G is the clock granularity, with a minimum of one second. When the timer expires the sender assumes heavy congestion: it sets ssthresh = max(FlightSize / 2, 2 × MSS), drops cwnd to one segment (the loss window) and begins slow start again. Each further timeout for the same segment doubles the RTO (exponential backoff).

Fast retransmit

Waiting for a timeout wastes time. A receiver that gets an out-of-order segment immediately repeats its last acknowledgement. When the sender sees **three duplicate ACKs** it concludes that one segment was lost while later ones arrived, and retransmits the missing segment at once, without waiting for the RTO.

Fast recovery (TCP Reno)

Duplicate ACKs mean data is still flowing, so collapsing to one segment would be an overreaction. In fast recovery the sender sets ssthresh to half the flight size and cwnd to ssthresh + 3 MSS (the three segments known to have left the network). Every further duplicate ACK inflates cwnd by one MSS; when the ACK for new data arrives, cwnd is deflated back to ssthresh and congestion avoidance resumes. TCP Tahoe, the earlier variant, had no fast recovery and always restarted from one segment.

Modern Congestion Control

CUBIC

CUBIC has been the default congestion control in Linux since kernel 2.6.19 (2006). Instead of growing one segment per RTT, it sets the window from the time t since the last loss: $W(t) = C(t - K)^3 + W_{max}$, where W_{max} is the window at the last loss, $C = 0.4$, and K is the time needed to climb back to W_{max}. On loss CUBIC multiplies the window by $\beta = 0.7$ rather than halving it. Because growth depends on elapsed time and not on the number of RTTs, flows with long round-trip times are not starved by flows with short ones.

BBR

BBR (Bottleneck Bandwidth and Round-trip propagation time), published by Google in 2016, does not treat loss as its main signal. It continuously estimates the bottleneck bandwidth and the minimum RTT of the path, paces packets at the estimated bandwidth and keeps about one bandwidth-delay product in flight. This keeps queues short and performs well on paths with random, non-congestive loss.

Explicit Congestion Notification

With ECN, a router that is starting to queue marks packets with the Congestion Experienced (CE) codepoint instead of dropping them. The receiver echoes the mark with the ECE flag and the sender, after reducing its window, confirms with the CWR flag. The flow slows down without losing a packet.

Exam note

Question 3 of the CS-342 midterm covers fast recovery; question 5 asks you to sketch the sawtooth.